Glossario dei termini tecnici

accoppiamento	Grado di dipendenza fra componenti software; dipendenze non necessarie rendono i cambiamenti più costosi.
artefatto	Prodotto intermedio o finale di un'attività di sviluppo, per esempio requisito, modello, codice o test.
backpressure	Controllo del flusso nel quale il consumatore comunica quanti elementi è pronto a ricevere.
classe	Modello che definisce stato e operazioni di oggetti dello stesso tipo.
coesione	Coerenza interna di dati e operazioni di un componente rispetto alla sua responsabilità.
Command–Query Separation	Separazione fra operazioni che modificano lo stato e operazioni che restituiscono informazioni senza modificarlo.
composition root	Punto dell'applicazione che costruisce e collega le istanze, fornendo le dipendenze ai componenti.
contratto	Insieme di operazioni e condizioni osservabili che un componente garantisce ai client.
DDD	Sviluppo guidato dalla comprensione del dominio applicativo, che orienta il modello e le responsabilità degli oggetti.
debito tecnico	Costo futuro dovuto a una soluzione rapida o poco curata che ostacola comprensione e modifica del software.
dependability	Capacità di offrire un servizio su cui sia possibile riporre fiducia giustificata; comprende aspetti di reliability, safety e security.
Dependency Injection	Tecnica con cui un oggetto riceve dall'esterno i propri collaboratori anziché costruirli direttamente.

design pattern	Soluzione specializzabile a un problema di progettazione ricorrente in un dato contesto.
DRY	Don't Repeat Yourself: evitare rappresentazioni duplicate della stessa conoscenza.
feature	Insieme di requisiti correlati che consente all'utente di raggiungere un obiettivo.
hook	Operazione con comportamento predefinito che una sottoclasse può ridefinire per personalizzare un punto di un algoritmo.
interfaccia	Contratto di operazioni visibile ai client; in Java è anche il costrutto linguistico che può dichiarare tale contratto.
iterazione	Ciclo di lavoro delimitato in cui si pianifica, realizza e verifica un incremento o una parte del prodotto.
KISS	Keep It Simple, Stupid: preferire soluzioni semplici da comprendere e gestire.
Law of Demeter	Principio che limita la conoscenza dei dettagli di oggetti lontani favorendo comunicazioni con collaboratori diretti.
manutenibilità	Facilità con cui il software può essere compreso, corretto, testato e modificato.
metrica	Misura di una proprietà del software usata per osservarne dimensione, struttura o andamento.
modello	Rappresentazione selettiva di aspetti del sistema, usata per comprenderlo, progettarlo o comunicarlo.
oggetto	Istanza di una classe, dotata di stato e operazioni.
oracle	Riferimento che consente a un test di stabilire se il risultato osservato è corretto.
polimorfismo	Proprietà per cui una stessa invocazione può produrre comportamenti diversi in base al tipo concreto del ricevente.

refactoring	Trasformazione che migliora la struttura interna preservando il comportamento osservabile.
registro dei prototipi	Associazione fra chiavi semantiche e oggetti già configurati da cui ottenere nuove istanze mediante copia.
requisito	Caratteristica, servizio o vincolo che il sistema deve soddisfare.
responsabilità	Compito coerente assegnato a un elemento del sistema; è anche un motivo di cambiamento.
risposta	Risultato o effetto prodotto dal sistema in seguito a una richiesta, un input o un evento.
SDD	Specification Driven Development: approccio che mette al centro le specifiche nel guidare anche l'implementazione assistita da IA.
stream	Sequenza di elementi elaborata da una pipeline; non è una collection che memorizza gli elementi.
test case	Caso di test con stato iniziale, input, oracle e risultato atteso.
test di regressione	Riesecuzione di test dopo una modifica per controllare che comportamenti già corretti restino tali.
UML	Unified Modeling Language: notazione grafica standardizzata per modellare struttura e comportamento di un sistema.
validazione	Controllo che il prodotto soddisfi bisogni e richieste dell'utente.
verifica	Controllo che gli artefatti siano costruiti correttamente rispetto a specifiche e progettazione.
YAGNI	You Ain't Gonna Need It: implementare le funzionalità richieste evitando estensioni puramente speculative.